

Chapter 3.

TRAINING DEEP NEURAL NETWORKS

PhD. Nguyễn Thị Khánh Tiên
tienntk@ut.edu.vn



Training Deep Neural Networks

Deep neural networks (DNNs) are computer models constructed from multiple layers of artificial neurons, mimicking how the human brain processes information. "Deep" refers to the number of hidden layers in the network. Networks with more hidden layers can learn more complex patterns from data.

Training DNNs is the process of teaching a computer model to perform a specific task using data.

The DNN Training Process:

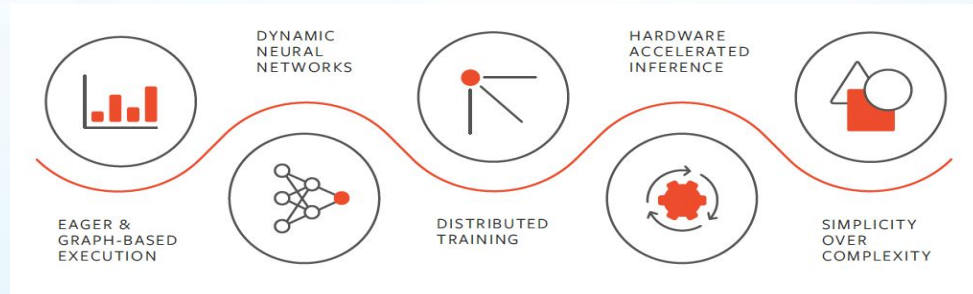
- **Data:** A large amount of labeled data is needed to train the model.
- **Algorithms:** The most common algorithm for training DNNs is backpropagation. This algorithm adjusts the network's weights to minimize the error between the predicted output and the actual output.
- **Optimization:** Optimization algorithms (e.g., Adam, SGD) are used to find the optimal weights.
- **Iteration:** The training process is often repeated many times (epochs) for the model to learn patterns more accurately.

Challenges in Training DNNs:

- **Overfitting:** The model learns the training data too well, but performs poorly on new data.
- **Vanishing/exploding gradients:** Problems that occur when gradients become too small or too large during backpropagation, making training difficult.
- **High computational requirements:** Training DNNs requires significant computing power, especially for large networks and large datasets.

Tools and Libraries:

- TensorFlow
- PyTorch
- Keras

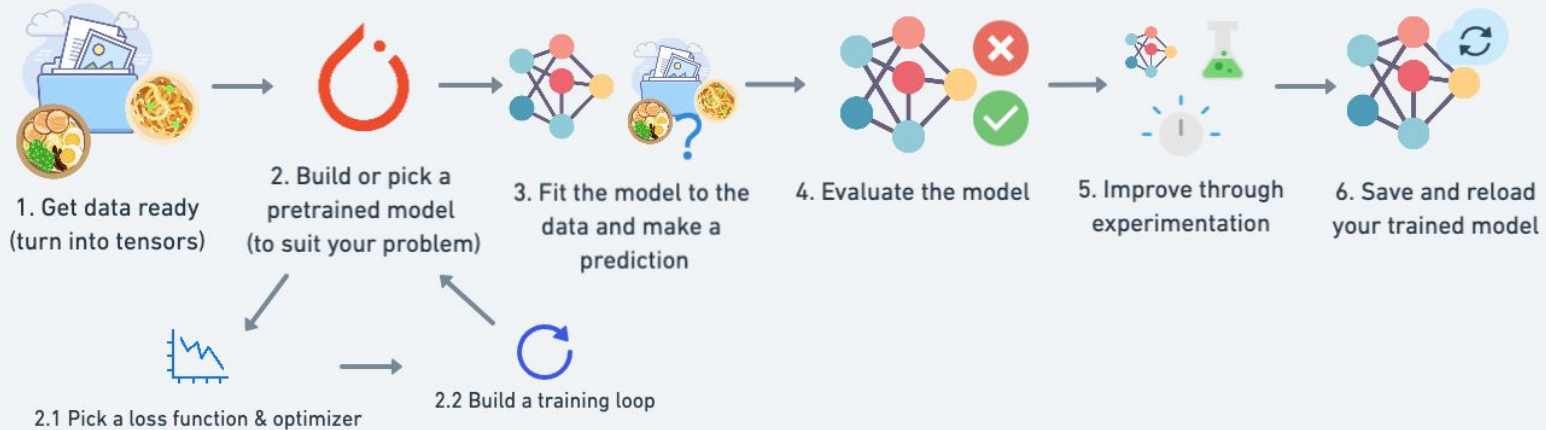


Training a DNN with PyTorch

Fundamental steps Steps in Training a DNN with PyTorch:

1. Prepare Your Data (Datasets and DataLoaders)
2. Define Your Model (Neural Network Architecture)
3. Choose a Loss Function and Optimizer
4. Write the Training Loop
5. Validation (Optional but Highly Recommended)
6. Save and Load Your Model (for later use)

A PyTorch Workflow



Prepare Data (Datasets and DataLoaders)

PyTorch provides **Dataset** and **DataLoader** classes to efficiently manage your data.

- **Dataset:** This class is responsible for:
 - Storing the samples and their corresponding labels.
 - Implementing `__len__` (returns the size of the dataset).
 - Implementing `__getitem__` (returns a sample and its label at a given index).
- You'll usually create a custom **Dataset** class that inherits from `torch.utils.data.Dataset`. You'll need to adapt this to your specific data format.
- **DataLoader:** This class is responsible for:
 - Iterating through the dataset in batches.
 - Shuffling the data (important for training).
 - Loading data in parallel using multiple workers (speeding up data loading).

```
1 import torch
2 from torch.utils.data import Dataset, DataLoader
3
4 class SimpleDataset(Dataset):
5     def __init__(self, data, labels):
6         self.data = data
7         self.labels = labels
8
9     def __len__(self):
10        return len(self.data)
11
12    def __getitem__(self, idx):
13        sample = self.data[idx]
14        label = self.labels[idx]
15        return sample, label
16
17 # Example Usage:
18 data = torch.randn(100, 10) # 100 samples, each 10 features
19 labels = torch.randint(0, 2, (100,)) # Binary classification labels (0 or 1)
20
21 dataset = SimpleDataset(data, labels)
22 dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
23
```

Data Preparation (Datasets and DataLoaders)

Example: For common datasets (like MNIST, CIFAR10, etc.), PyTorch `torchvision.datasets` module provides pre-built datasets. You can download and use them directly.

```
1  from torchvision import datasets, transforms
2
3  # Define transformations to preprocess the data (e.g., normalization)
4  transform = transforms.Compose([
5      transforms.ToTensor(), # Convert PIL image to tensor
6      transforms.Normalize((0.1307,), (0.3081,)) # Normalize for MNIST
7  ])
8
9  # Load MNIST dataset
10 train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
11 train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=True)
12
13 test_dataset = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
14 test_dataloader = DataLoader(test_dataset, batch_size=1000, shuffle=False) # No need to shuffle test data
```

Define Model (Neural Network Architecture)

You'll use **torch.nn.Module** to define your DNN architecture. This involves creating classes that inherit from **nn.Module** and defining:

- **__init__(self)**: Where you define the layers of your network (e.g., **nn.Linear**, **nn.Conv2d**, **nn.ReLU**, **nn.MaxPool2d**, etc.).
- **forward(self, x)**: Where you define the forward pass of your network, specifying how data flows through the layers.

Example: Simple Feedforward DNN for Classification

```
1 import torch.nn as nn
2 import torch.nn.functional as F # Functional interface for activation functions
3
4 class SimpleDNN(nn.Module):
5     def __init__(self, input_size, hidden_size, num_classes):
6         super(SimpleDNN, self).__init__()
7         self.fc1 = nn.Linear(input_size, hidden_size) # First fully connected layer
8         self.relu = nn.ReLU() # ReLU activation function
9         self.fc2 = nn.Linear(hidden_size, num_classes) # Output layer
10
11     def forward(self, x):
12         out = self.fc1(x)
13         out = self.relu(out)
14         out = self.fc2(out)
15         return out
16
17 # Example instantiation:
18 input_size = 10 # Example input feature size
19 hidden_size = 50
20 num_classes = 2 # Binary classification
21 model = SimpleDNN(input_size, hidden_size, num_classes)
22
```

Choose a Loss Function and Optimizer

Loss Function (Criterion): Measures how well your model is performing. You need to choose a loss function appropriate for your task (classification, regression, etc.). Common options include:

- **nn.CrossEntropyLoss** (for multi-class classification)
- **nn.BCELoss** or **nn.BCEWithLogitsLoss** (for binary classification)
- **nn.MSELoss** (for regression)

Optimizer: Algorithm that updates the model's parameters (weights and biases) to minimize the loss function. Popular optimizers:

- **torch.optim.SGD** (Stochastic Gradient Descent)
- **torch.optim.Adam** (Adam optimizer - often a good default)
- **torch.optim.RMSprop**

```
1 import torch.optim as optim
2
3 # Loss function for multi-class classification (if num_classes > 2) or binary (if num_classes == 2)
4 criterion = nn.CrossEntropyLoss() # Or nn.BCEWithLogitsLoss if binary with sigmoid output
5
6 # Optimizer (Adam is a good general choice)
7 optimizer = optim.Adam(model.parameters(), lr=0.001) # lr is the learning rate
```


Write the Training Loop

It typically involves iterating over your data in epochs and batches.

Basic Training Loop Structure:

```
1 num_epochs = 10 # Example number of epochs
2
3 for epoch in range(num_epochs):
4     for batch_idx, (data, labels) in enumerate(train_dataloader):
5         # 1. Move data to device (GPU if available, otherwise CPU)
6         data = data # .to(device) if using GPU
7         labels = labels # .to(device)
8
9         # 2. Zero gradients (important before each batch)
10        optimizer.zero_grad()
11
12        # 3. Forward pass: compute model outputs
13        outputs = model(data)
14
15        # 4. Calculate loss
16        loss = criterion(outputs, labels)
17
18        # 5. Backward pass: compute gradients
19        loss.backward()
20
21        # 6. Optimization step: update weights
22        optimizer.step()
23
24        # Print training statistics (optional, but good for monitoring)
25        if (batch_idx+1) % 10 == 0: # Print every 10 batches
26            print(f'Epoch: {epoch+1}/{num_epochs}')
27            print(f'Batch [{batch_idx+1}/{len(train_dataloader)}], Loss: {loss.item():.4f}')
28
29 print("Training finished!")
```


Explanation of the Training Loop Steps

1. **Move Data to Device:** If you're using a GPU for faster training (highly recommended for DNNs), you need to move your data and model to the GPU device. You would use `.to(device)` for this (where `device` is `'cuda'` if CUDA is available, or `'cpu'`).
2. **Zero Gradients:** `optimizer.zero_grad()` clears the gradients from the previous batch. PyTorch accumulates gradients by default, so you must zero them before each new batch.
3. **Forward Pass:** `outputs = model(data)` feeds the input data (`data`) through your model to get the model's predictions (`outputs`).
4. **Calculate Loss:** `loss = criterion(outputs, labels)` calculates the loss between the model's outputs (`outputs`) and the true labels (`labels`) using your chosen loss function.
5. **Backward Pass:** `loss.backward()` computes the gradients of the loss with respect to all the model's parameters (weights and biases). This is the backpropagation step.
6. **Optimization Step:** `optimizer.step()` updates the model's parameters based on the calculated gradients and the optimization algorithm you've chosen (e.g., Adam).

Validation

It's crucial to evaluate your model's performance on a separate validation dataset during training. Monitor for overfitting (when the model performs well on the training data but poorly on unseen data).

- Choose the best model during training (e.g., based on validation accuracy).
- Tune hyperparameters (learning rate, regularization, etc.).

Key Points for Validation:

- **model.eval():** Puts the model in evaluation mode. This is essential for layers like Batch Normalization and Dropout, which behave differently during training and evaluation.
- **torch.no_grad():** Disables gradient calculations, making validation faster and using less memory.
- **model.train():** After validation, switch the model back to training mode for the next epoch of training.

Validation

Example: Validation Loop (to be added within the training loop):

```
1 # ... inside the training loop, after the epochs loop
2
3 def validate(model, dataloader, criterion, device):
4     model.eval() # Set model to evaluation mode (important for batch normalization, dropout, etc.)
5     total_loss = 0
6     correct_predictions = 0
7     total_samples = 0
8
9     with torch.no_grad(): # Disable gradient calculations during validation (for efficiency)
10         for data, labels in dataloader:
11             data = data.to(device)
12             labels = labels.to(device)
13             outputs = model(data)
14             loss = criterion(outputs, labels)
15             total_loss += loss.item()
16
17             _, predicted_labels = torch.max(outputs.data, 1) # For classification, get predicted class indices
18             total_samples += labels.size(0)
19             correct_predictions += (predicted_labels == labels).sum().item()
20
21     avg_loss = total_loss / len(dataloader)
22     accuracy = 100 * correct_predictions / total_samples
23     return avg_loss, accuracy
24
25 # ... Inside the epochs loop, after each epoch (or after certain intervals)
26
27 # After each epoch (or less frequently):
28 val_loss, val_accuracy = validate(model, test_dataloader, criterion, device) # Using test_dataloader as validation set here for simplicity
29 print(f'Epoch [{epoch+1}/{num_epochs}], Validation Loss: {val_loss:.4f}, Validation Accuracy: {val_accuracy:.2f}%')
30
31 model.train() # Set model back to training mode after validation
```


Save and Load Your Model

After training, you'll want to **save your model's learned parameters (weights and biases)** so you can reuse it later without retraining

```
1 torch.save(model.state_dict(), 'model.pth') # Save only the model's parameters
2 # Or to save the entire model structure (less common for deployment)
3 torch.save(model, 'full_model.pth')
```

Loading the Model

```
1 # 1. Instantiate your model class (structure must be the same as when you saved it)
2 loaded_model = SimpleDNN(input_size, hidden_size, num_classes)
3
4 # 2. Load the saved state dictionary
5 loaded_model.load_state_dict(torch.load('model.pth'))
6
7 # 3. Put the model in evaluation mode if you're only doing inference
8 loaded_model.eval()
```

Visualizing Models, Data, and Training with TensorBoard

TensorBoard is a powerful tool for visualizing various aspects of your PyTorch models, data, and training process.

Tutorial → https://pytorch.org/tutorials/intermediate/tensorboard_tutorial.html

Advantages:

- Real-time visualization: See your training progress as it happens.
- Easy debugging: Identify issues like vanishing gradients or overfitting.
- Model comparison: Compare different models or hyperparameters.
- Data exploration: Understand your data through visualizations.
- Share results: Easily share your training progress with others.

1. Installation

- Make sure you have **tensorboard** and **torch** installed:

`pip install tensorboard torch torchvision`

2. Setting Up TensorBoard in PyTorch

- Import **SummaryWriter**: This class is responsible for writing data to TensorBoard logs.
`from torch.utils.tensorboard import SummaryWriter`
- Create a **SummaryWriter** instance: Specify the directory where you want to store the logs.
`writer = SummaryWriter('runs/experiment_1') # 'runs/experiment_1' is the log directory`

Visualizing Models, Data, and Training with TensorBoard

3. Visualizing Scalars (Loss, Accuracy, etc.)

Add scalar values: Use `writer.add_scalar()` to log scalar values like loss, accuracy, or learning rate at each training step.

4. Visualizing Model Architecture

Add graph: Use `writer.add_graph()` to visualize the structure of your model. You need to provide a dummy input to trace the model.

5. Visualizing Images

Add images: Use `writer.add_image()` to visualize images from your dataset or predictions.

6. Visualizing Histograms

Add histograms: Use `writer.add_histogram()` to visualize the distribution of weights or activations.

7. Visualizing Embeddings

Add embeddings: Use `writer.add_embedding()` to visualize high-dimensional data (e.g., word embeddings) in a lower-dimensional space.

8. Running TensorBoard

Open TensorBoard: In your terminal, navigate to the directory containing your logs and run:

Bash `tensorboard --logdir=runs`

Access TensorBoard: Open your web browser and go to <http://localhost:6006/>.

Visualizing Models, Data, and Training with TensorBoard

```
1 import torch
2 import torch.nn as nn
3 import torchvision.models as models
4 ▶ Launch TensorBoard Session
5 from torch.utils.tensorboard import SummaryWriter
6 import torchvision.transforms as transforms
7 import torchvision.datasets as datasets
8 import torchvision.utils as vutils
9
10 # Create a SummaryWriter
11 writer = SummaryWriter('runs/resnet18_experiment')
12
13 # Load a pre-trained model
14 model = models.resnet18(pretrained=False)
15
16 # Create a dummy input
17 dummy_input = torch.rand(1, 3, 224, 224)
18 writer.add_graph(model, dummy_input)
```

```
18
19 # Create example data
20 transform = transforms.Compose([transforms.ToTensor()])
21 dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
22 dataloader = torch.utils.data.DataLoader(dataset, batch_size=64)
23
24 # Example training loop (simplified)
25 for epoch in range(2):
26     for i, (images, labels) in enumerate(dataloader):
27         # ... (Training code) ...
28         loss = torch.rand(1) # example loss.
29         writer.add_scalar('Loss/train', loss, epoch * len(dataloader) + i)
30         if i % 100 == 0:
31             grid = vutils.make_grid(images)
32             writer.add_image('MNIST_images', grid, epoch * len(dataloader) + i)
33
34 for name, param in model.named_parameters():
35     writer.add_histogram(name, param, 1)
36
37 # Close the writer
38 writer.close()
39
40 #Run in terminal: tensorboard --logdir=runs
```


Vanishing and Exploding Gradients

Vanishing Gradients:

- Occurs when gradients become too small during backpropagation, causing weights in early layers to update ineffectively.
- Common with activation functions like sigmoid or tanh.

Exploding Gradients:

- Occurs when gradients become too large, leading to unstable weight updates and potentially breaking the training process.
- Common in deep networks or recurrent neural networks (RNNs).

Solutions:

- Use activation functions like ReLU or its variants (e.g., LeakyReLU) to mitigate vanishing gradients.
- Apply gradient clipping to limit excessively large gradients.
- Careful weight initialization (covered later).
- use batch normalization.

Weight Initialization

Purpose: Initialize weights appropriately to promote stable and efficient training.

Common Methods:

- *Random Initialization:* Initialize weights from a random distribution (e.g., normal or uniform).
- *Xavier/Glorot Initialization:* Initializes weights based on the number of inputs and outputs of a layer, balancing gradients during backpropagation.
- *He Initialization:* Similar to Xavier, but optimized for ReLU activation functions.

Implementation in PyTorch:

- Use `torch.nn.init` to initialize weights.

```
1 import torch.nn as nn
2 import torch.nn.init as init
3
4 linear_layer = nn.Linear(10, 5)
5 init.xavier_uniform_(linear_layer.weight) #Xavier init
```

Activation Functions & Batch Normalization

Activation Functions

- **Purpose:** Introduce non-linearity into neural networks, enabling them to learn complex relationships.
- **Common Activation Functions:**
 - **ReLU (Rectified Linear Unit):** $\text{ReLU}(x) = \max(0, x)$. Simple, efficient, and mitigates vanishing gradients.
 - **Sigmoid:** value between 0 and 1, often used in binary classification.
 - **Tanh (Hyperbolic Tangent):** value between -1 and 1.
 - **LeakyReLU:** improves ReLU by allowing a small gradient when inputs are negative.
- **Implementation in PyTorch:** Use layers in **torch.nn** (e.g., **nn.ReLU**, **nn.Sigmoid**, **nn.Tanh**, **nn.LeakyReLU**).

```
1 import torch.nn as nn
2
3 model = nn.Sequential(
4     nn.Linear(784, 128),
5     nn.BatchNorm1d(128),
6     nn.ReLU(),
7     nn.Linear(128, 10)
8 )
```

Batch Normalization

- **Purpose:** Normalize the outputs of layers in a network to mitigate vanishing and exploding gradients, and accelerate training.
- **How it Works:**
 - Normalizes layer outputs to a standard distribution (mean = 0, variance = 1).
 - Applies scale and shift parameters to allow the network to learn the optimal distribution.
- **Implementation in PyTorch:**
 - Use **nn.BatchNorm1d** (for 1D data), **nn.BatchNorm2d** (for 2D data), or **nn.BatchNorm3d** (for 3D data).

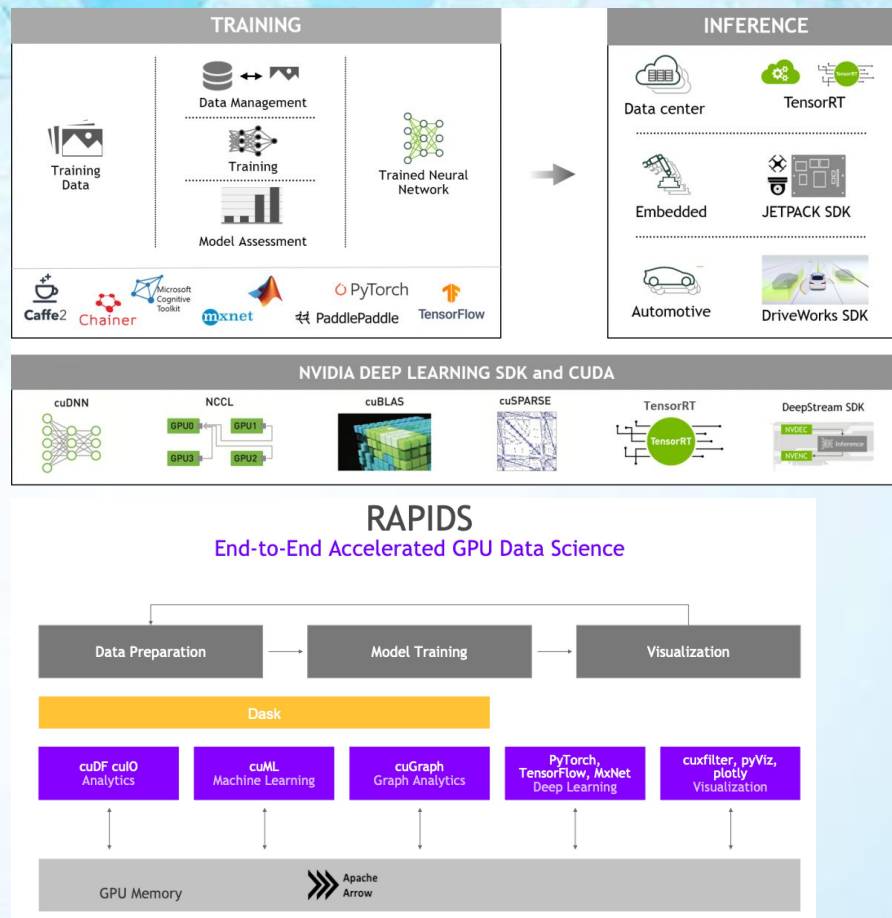
Important Considerations:

Device (CPU vs. GPU): For real-world DNN training, using a GPU is crucial for speed. You'll need to move your data and model to the GPU device using `.to(device)` where `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')`.

Hyperparameter Tuning: The learning rate, batch size, number of epochs, network architecture (number of layers, hidden units), regularization parameters, etc., are hyperparameters. You'll typically need to experiment and tune these to find the best values for your specific problem.

More Complex Models: For image data, you would use Convolutional Neural Networks (CNNs) with layers like `nn.Conv2d`, `nn.MaxPool2d`, etc. For sequence data (text, time series), you might use Recurrent Neural Networks (RNNs) or Transformers.

Ref → <https://www.nvidia.com/en-au/glossary/pytorch/>



Reusing Pre-trained Layers (Transfer Learning)

Purpose: To leverage knowledge from pre-trained models on large datasets (e.g., ImageNet) to improve performance on your smaller dataset.

Steps:

1. *Load the Pre-trained Model:* PyTorch provides many pre-trained models in **torchvision.models**.
2. *Freeze Layers:* Freeze the layers of the pre-trained model to prevent them from updating their weights during retraining.
3. *Replace or Add Layers:* Replace the final classification layer of the model to match the number of classes in your dataset. You can also add additional layers for customization.
4. *Retrain:* Retrain the model on your dataset. Only the weights of the unfrozen layers will be updated.

```
1 import torch
2 import torchvision.models as models
3 import torch.nn as nn
4
5 # Load pre-trained ResNet-18
6 model = models.resnet18(pretrained=True)
7
8 # Freeze all parameters in the model
9 for param in model.parameters():
10     param.requires_grad = False
11
12 # Replace the final fully connected layer
13 num_fts = model.fc.in_features
14 model.fc = nn.Linear(num_fts, 10) # 10 is your number of classes
15
16 # Retrain the model
17 # ... (training code)
```

Fine-tuning

Purpose: To refine the weights of a pre-trained model on your dataset for improved performance.

Steps:

1. *Load the Pre-trained Model:* Similar to transfer learning.
2. *Unfreeze Layers:* Unfreeze some of the final layers of the model to allow them to update their weights.
3. *Retrain:* Retrain the model on your dataset.

Transfer learning and fine-tuning are particularly useful when you have a small dataset.

```
1 import torch
2 import torchvision.models as models
3 import torch.nn as nn
4
5 # Load pre-trained ResNet-18
6 model = models.resnet18(pretrained=True)
7
8 # Unfreeze some of the final layers
9 for param in model.layer4.parameters():
10     param.requires_grad = True
11
12 # Replace the final fully connected layer
13 num_ftrs = model.fc.in_features
14 model.fc = nn.Linear(num_ftrs, 10)
15
16 # Retrain the model
17 # ... (training code)
```

Regularization Techniques

Purpose: To prevent overfitting by adding a penalty to the loss function. Experiment with different hyperparameters (e.g., learning rate, dropout rate, regularization strength) to find the best values for your dataset.

Dropout

Purpose: Randomly deactivates neurons during training to prevent co-adaptation of neurons.

Implementation in PyTorch: Use the **nn.Dropout** layer to add dropout layers to your model. Dropout is only applied during training.

```
1 import torch.nn as nn
2 import torch.optim as optim
3
4 # Model with dropout
5 model = nn.Sequential(
6     nn.Linear(784, 128),
7     nn.ReLU(),
8     nn.Dropout(0.5), # Dropout with probability 0.5
9     nn.Linear(128, 10)
10 )
11
12 # Optimizer with L2 regularization
13 optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
```

L1 and L2 Regularization:

L1 Regularization (Lasso): Adds the sum of the absolute values of the weights to the loss function.

- Encourages sparse weights (many weights equal to zero).
- Useful for feature selection.

L2 Regularization (Ridge): Adds the sum of the squared values of the weights to the loss function.

- Encourages small weights.
- Reduces the impact of outliers.

Implementation in PyTorch:

- Use the **weight_decay** parameter in the optimizer to apply L2 regularization.
- For L1 regularization, you need to manually add the L1 penalty to the loss function.

Monitor the model's performance on a validation dataset to detect overfitting and adjust regularization techniques accordingly.

Practice 2 - Hands-on practice with pre-trained neural network architectures

Exercises:

- 1) Experiment with different pre-trained models (VGG, DenseNet, etc.).
- 2) Adjust the hyperparameters (learning rate, batch size, etc.).
- 3) Use TensorBoard to monitor the training process.

Objective:

- Become familiar with loading and using pre-trained models from **torchvision.models**.
- Understand how to adapt and fine-tune pre-trained models for specific tasks.

Steps:

1. Environment Setup:

- Ensure you have PyTorch and **torchvision** installed.
- If not, install them using: **pip install torch torchvision**

2. Load a Pre-trained Model:

- **torchvision.models** provides various pre-trained models like ResNet, VGG, DenseNet, etc.
- **pretrained=True** loads weights pre-trained on the ImageNet dataset.

3. Explore the Model Architecture:

- Print the model to examine its structure.
- Identify the layers and parameters of the model.

4. Adapt the Model for a Specific Task:

- Typically, the final layer of a pre-trained model (the classification layer) needs to be replaced to match the number of classes in your dataset.
- You can freeze earlier layers to prevent them from updating weights during retraining (transfer learning).
- Or you can unfreeze a few of the final layers to fine-tune the model.

5. Prepare the Data:

- Load and prepare your dataset.
- Use **torchvision.transforms** to preprocess the data.
- Use **torch.utils.data.DataLoader** to create data batches.

6. Train the Model:

- Define the loss function and optimizer.
- Write the training loop to train the model on your dataset.

7. Evaluate the Model: Evaluate the model's performance on a test dataset.

Example (using the CIFAR-10 dataset):

```
1 import torch
2 import torchvision
3 import torchvision.transforms as transforms
4 import torch.nn as nn
5 import torch.optim as optim
6
7 # 1. Load ResNet-18
8 resnet18 = torchvision.models.resnet18(pretrained=True)
9
10 # 2. Change the final layer
11 num_ftrs = resnet18.fc.in_features
12 resnet18.fc = nn.Linear(num_ftrs, 10)
13
14 # 3. Prepare the CIFAR-10 data
15 transform = transforms.Compose([
16     transforms.ToTensor(),
17     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
18 ])
19
20 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
21                                         download=True, transform=transform)
22 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
23                                         shuffle=True, num_workers=2)
24
```

```
24
25 # 4. Loss function and optimizer
26 criterion = nn.CrossEntropyLoss()
27 optimizer = optim.SGD(resnet18.fc.parameters(), lr=0.001, momentum=0.9)
28
29 # 5. Train the model
30 for epoch in range(2): # loop over the dataset multiple times
31
32     running_loss = 0.0
33     for i, data in enumerate(trainloader, 0):
34         inputs, labels = data
35
36         optimizer.zero_grad()
37
38         outputs = resnet18(inputs)
39         loss = criterion(outputs, labels)
40         loss.backward()
41         optimizer.step()
42
43         running_loss += loss.item()
44         if i % 2000 == 1999: # print every 2000 mini-batches
45             print('[%d, %5d] loss: %.3f' %
46                   (epoch + 1, i + 1, running_loss / 2000))
47             running_loss = 0.0
48
49 print('Finished Training')
```

Q & A